

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ"

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

Лабораторная работа № 2

по дисциплине "Проектирование интерфейсов и системный анализ"
на тему "Проектирование веб-интерфейса и создание структуры базы
данных. Подключение ADO.NET и реализация CRUD-операций."

Факультет: ФПМИ

Группа: ПМИ-32

Студенты: Весёлый Д., Ворончук И., Лыкова М.

Бригада №2

Преподаватели: Кутузова И. А.

НОВОСИБИРСК 2026

Цель: формирование навыков создания веб-интерфейса в соответствии с принципами проектирования пользовательского интерфейса и навыков реализации бизнес-логики windows-приложения и веб-интерфейса.

Вариант	Тема
2	Книжный каталог

Цель работы: Формирование навыков реализации бизнес-логики веб-приложения и подключения к СУБД MS SQL Server средствами ADO.NET. Доработка интерфейса, реализованного в лабораторной работе №1, с целью обеспечения полноценной работы всех вариантов использования из лабораторной работы №1.

1.1. Компоненты ADO.NET, использованные в проекте:

В проекте были применены следующие компоненты ADO.NET:

- SqlConnection используется для управления соединением с базой данных. В коде соединение открывается и закрывается с помощью конструкции using, что гарантирует освобождение ресурсов: `using (var conn = new SqlConnection(connStr)).`
- SqlCommand отвечает за выполнение запросов и хранимых процедур. Пример использования: `new SqlCommand("sp_SearchBooks", conn).`
- SqlParameter обеспечивает параметризацию запросов, что защищает приложение от SQL-инъекций. Пример: `cmd.Parameters.AddWithValue("@Email", email).`
- SqlDataAdapter используется для заполнения объектов DataTable результатами выборки из базы данных через метод `Fill(dataTable).`
- DataTable хранит данные в памяти для последующей привязки к элементам управления интерфейса, например: `gvBooks.DataSource = dt.`
- CommandType.StoredProcedure указывает, что выполняемая команда является хранимой процедурой, а не текстовым SQL-запросом: `cmd.CommandType = CommandType.StoredProcedure.`

- ParameterDirection.Output позволяет получать выходные параметры из хранимых процедур: param.Direction = ParameterDirection.Output.
- SqlTransaction обеспечивает атомарность операций через механизмы BEGIN TRANSACTION, COMMIT и ROLLBACK, что критично для операций бронирования и списания.

1.2. Архитектура слоя данных

Архитектура взаимодействия с данными построена по трёхуровневой схеме. Страницы приложения (файлы .aspx.cs) обращаются к статическому классу DataStore, который является единой точкой доступа ко всем операциям с данными. Класс DataStore делегирует выполнение низкоуровневых операций классу DBHelper, который инкапсулирует работу с ADO.NET (создание соединений, выполнение команд, обработка результатов). На нижнем уровне находится СУБД MS SQL Server (LocalDB), содержащая таблицы и хранимые процедуры.

1.3. Пример кода: выполнение хранимой процедуры с параметрами

```
public static int RegisterUser(string fullName, string email,
string password)
{
    using (var conn = new
SqlConnection(DbHelper.ConnectionString))
    {
        conn.Open();
        using (var cmd = new SqlCommand("sp_RegisterUser", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("@FullName", fullName);
            cmd.Parameters.AddWithValue("@Email", email);

            var outId = new SqlParameter("@UserID", SqlDbType.Int)
                { Direction = ParameterDirection.Output };
            var outErr = new SqlParameter("@ErrorMessage",
SqlDbType.NVarChar, 255)
                { Direction = ParameterDirection.Output };
            cmd.Parameters.Add(outId);
            cmd.Parameters.Add(outErr);
```

```

        cmd.ExecuteNonQuery();

        return (int)outId.Value; // -1 = ошибка, >0 = ID
пользователя
    }
}
}

```

2. Реализация вариантов использования из лабораторной работы №1

Сценарий 1: Регистрация пользователя

Предусловие: посетитель не имеет учётной записи в системе.

Используемые объекты БД: таблица Users, хранимая процедура sp_RegisterUser.

Шаги выполнения:

1. Посетитель нажимает кнопку «Регистрация» на главной странице - система перенаправляет на страницу Register.aspx.
2. Система отображает форму с полями: ФИО, email, пароль, подтверждение пароля.
3. Посетитель вводит данные. Клиентская валидация проверяет обязательность заполнения всех полей (RequiredFieldValidator), корректность формата email (RegularExpressionValidator) и совпадение паролей (CompareValidator).
4. При отправке формы Register.aspx.cs вызывает DataStore.UserExists(email), который выполняет прямой SQL-запрос к таблице Users: SELECT 1 FROM Users WHERE Email = @Email. Поле Email имеет ограничение UNIQUE NOT NULL - гарантия уникальности обеспечивается как на уровне приложения, так и на уровне SQL Server.
5. Если email уже занят, система отображает панель с предложением восстановить пароль и ссылкой на ForgotPassword.aspx.
6. Если email свободен - вызывается DataStore.RegisterUser(fullName, email, password), который через DBHelper.ExecuteNonQuery передаёт управление хранимой процедуре sp_RegisterUser. Процедура повторно проверяет уникальность email на уровне БД (защита от race condition), выполняет INSERT INTO Users (FullName, Email, PasswordHash), возвращает новый UserID через SCOPE_IDENTITY() в OUTPUT-параметр.
7. DataStore.GetUserByEmail(email) загружает полную запись нового пользователя из Users (включая MaxBookings, Role, CreatedAt),

объект помещается в Session["User"].

8. Пользователь перенаправляется на Default.aspx.

Расширение восстановление пароля:

Пользователь переходит на ForgotPassword.aspx.

DataStore.GetUserByEmail(email) выполняет SELECT по таблице Users.

После ввода нового пароля DataStore.ResetPassword(userId, newPassword) выполняет:

```
UPDATE Users SET PasswordHash = @Hash WHERE UserID = @UserID
```

Сценарий 2: Добавление новой книги

Предусловие: Пользователь авторизован с ролью Admin.

Шаги выполнения:

1. Администратор выбирает в меню пункт “Добавить книгу” - система открывает страницу Admin/AddBook.aspx. Доступ ограничен на уровне Admin/Web.config.
2. Система отображает форму с полями: название, автор, ISBN, издательство, год издания (диапазон 1000–2026), количество страниц, жанр, количество экземпляров.
3. Администратор заполняет форму. Серверная валидация проверяет корректность числовых полей (RangeValidator, int.TryParse).
4. AddBook.aspx.cs вызывает DataStore.FindDuplicatesByISBN(isbn), который выполняет запрос:

```
SELECT BookID, Title, Author, ISBN, TotalCount, AvailableCount  
FROM Books WHERE ISBN = @ISBN AND IsArchived = 0
```

5. Если дубликат найден, система отображает список существующих книг с таким ISBN и панель подтверждения. При подтверждении добавления экземпляров вызывается sp_AddBook, которая обнаруживает совпадение по ISBN и выполняет:

```
UPDATE Books  
SET TotalCount      = TotalCount      + @Count,  
    AvailableCount = AvailableCount + @Count  
WHERE ISBN = @ISBN;
```

Новая запись в Books не создаётся - увеличивается количество экземпляров существующей книги.

6. Если дубликатов нет - DataStore.AddBook(book) вызывает sp_AddBook, которая выполняет:

```

INSERT INTO Books
    (Title, Author, ISBN, Publisher, Year, Pages, Genre,
     TotalCount, AvailableCount, CoverImageURL)
VALUES
    (@Title, @Author, @ISBN, @Publisher, @Year, @Pages, @Genre,
     @Count, @Count, @CoverImageURL);

```

Книге автоматически присваивается статус “В наличии” (AvailableCount = TotalCount). Новый BookID возвращается через SCOPE_IDENTITY().

7. После добавления книга немедленно попадает в результаты поиска благодаря индексу IX_Books_Search ON Books(Title, Author, Genre, Year).
8. Система отображает сообщение об успешном добавлении и очищает форму.

Сценарий 3: Поиск книги

Предусловие: Пользователь находится на главной странице.

Используемые объекты БД: таблица Books, хранимая процедура sp_SearchBooks, индекс IX_Books_Search.

Шаги выполнения:

1. Пользователь открывает Default.aspx. При загрузке страницы вызывается DataStore.SearchBooks(null, null, null, null) - sp_SearchBooks без фильтров возвращает все книги с IsArchived = 0, отсортированные по Title.
2. Пользователь заполняет одно или несколько полей фильтра: название, автор, жанр (выпадающий список), год издания.
3. Поле года проверяется компонентом RangeValidator на диапазон 1000–2026. Это дублирует ограничение CHECK в таблице Books.
4. Пользователь нажимает кнопку "Найти". Default.aspx.cs вызывает DataStore.SearchBooks(title, author, genre, year), который через DBHelper.ExecuteNonQuery вызывает sp_SearchBooks. Процедура применяет параметры через оператор LIKE с NULL-защитой:

```

SELECT BookID, Title, Author, ISBN, Publisher, Year, Genre,
       AvailableCount, TotalCount
FROM Books
WHERE IsArchived = 0
      AND (@Title IS NULL OR Title LIKE '%' + @Title + '%')
      AND (@Author IS NULL OR Author LIKE '%' + @Author + '%')
      AND (@Genre IS NULL OR Genre = @Genre)

```

```

AND (@YearFrom IS NULL OR Year >= @YearFrom)
AND (@YearTo IS NULL OR Year <= @YearTo)
ORDER BY Title;

```

Параметры передаются через SqlParameter - SQL-инъекции исключены.

Поиск ускоряется индексом IX_Books_Search.

5. Результаты поиска отображаются в GridView с колонками: название, автор, год, жанр, доступность, действия. Статус «В наличии» / «Забронировано» определяется значением AvailableCount из таблицы Books.
6. Если результаты отсутствуют, система отображает сообщение "Данные не обнаружены".
7. Пользователь нажимает «Подробнее» - DataStore.GetBookById(id) выполняет точечный SELECT по первичному ключу BookID:

```

SELECT BookID, Title, Author, ISBN, Publisher, Year, Pages,
        Genre, TotalCount, AvailableCount, IsArchived, CreatedAt
FROM Books WHERE BookID = @BookID

```

Система перенаправляет на BookDetails.aspx?id=N.

Сценарий 4: Бронирование книги

Предусловие: Пользователь авторизован, книга имеет статус "В наличии".

Используемые объекты БД: таблицы Books и BookReservations, хранимая процедура sp_CreateBooking, индексы IX_Reservations_User и IX_Reservations_Book.

Шаги выполнения:

1. На странице каталога (Default.aspx) или карточки книги (BookDetails.aspx) пользователь нажимает кнопку "Забронировать".
2. Система проверяет Session["User"]. Если пользователь не авторизован - перенаправление на Login.aspx?returnUrl=Booking.aspx%3Fid%3DN. После авторизации - автоматический возврат к бронированию.
3. Booking.aspx загружает данные о книге через DataStore.GetBookById(bookId) и число активных броней пользователя через DataStore.GetUserActiveBookingsCount(userId):

```

SELECT COUNT(*) AS Cnt FROM BookReservations
WHERE UserID = @UserID AND Status = 'Active'

```

Запрос ускоряется индексом IX_Reservations_User(UserID, Status).

4. Расширение 5.a - книга недоступна: если AvailableCount = 0, на BookDetails.aspx вместо “Забронировать” отображается “Встать в очередь”. DataStore.AddToQueue(userId, bookId) проверяет отсутствие дубликата в очереди и выполняет:

```
INSERT INTO BookReservations (UserID, BookID, ExpiresAt, Status)
VALUES (@UserID, @BookID, DATEADD(DAY, 30, GETDATE()), 'Queued')
```

5. Расширение 6.a - превышен лимит: Если количество активных броней пользователя достигло значения MaxBookings, система отображает сообщение об ошибке, бронь не создаётся.
6. При успешных проверках отображается страница подтверждения.
7. Пользователь нажимает “Подтвердить бронь”. DataStore.CreateBooking(userId, bookId) вызывает хранимую процедуру sp_CreateBooking, которая выполняет операции в транзакции:

```
BEGIN TRANSACTION;
BEGIN TRY
    INSERT INTO BookReservations (UserID, BookID, ExpiresAt)
    VALUES (@UserID, @BookID, DATEADD(DAY, 3, GETDATE()));
    UPDATE Books
    SET AvailableCount = AvailableCount - 1
    WHERE BookID = @BookID;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    SET @ErrorMessage = ERROR_MESSAGE();
END CATCH
```

Транзакция гарантирует: либо оба изменения применяются вместе, либо ни одно - целостность данных сохраняется даже при сбое.

8. Система отображает подтверждение с датой окончания брони.
9. Бронь отображается в профиле (UserProfile.aspx). При отмене пользователем DataStore.CancelBooking(reservationId, userId) выполняет:

```
UPDATE BookReservations SET Status = 'Cancelled'
WHERE ReservationID = @ReservationID AND UserID = @UserID AND
Status = 'Active';
```

```
UPDATE Books SET AvailableCount = AvailableCount + 1
```



```
WHERE BookID = (SELECT BookID FROM BookReservations
                WHERE ReservationID = @ReservationID);
```

10. Автоматическое истечение броней: при каждой загрузке Default.aspx вызывается DataStore.ExpireOldBookings():

```
UPDATE Books
SET AvailableCount = AvailableCount +
  (SELECT COUNT(*) FROM BookReservations r2
   WHERE r2.BookID = Books.BookID
     AND r2.Status = 'Active'
     AND r2.ExpiresAt < GETDATE());

UPDATE BookReservations SET Status = 'Expired'
WHERE Status = 'Active' AND ExpiresAt < GETDATE();
```

Сценарий 5: Списание книги

Предусловие: Пользователь авторизован с ролью Admin.

Используемые объекты БД: таблицы Books, BookReservations, WriteOffRecords, хранимая процедура sp_WriteOffBook.

Шаги выполнения:

1. Администратор переходит на страницу WriteOff.aspx ("Списание книг").
2. Система отображает каталог книг с возможностью фильтрации по названию, автору, жанру.
3. Администратор выбирает книгу, указывает причину списания из выпадающего списка, вводит количество экземпляров и нажимает "Списать".
4. Код страницы WriteOff.aspx.cs вызывает метод DataStore.WriteOffBook(bookId, reason, count, out writtenOff, adminId).
5. Расширение 4.a - активные брони: если count > AvailableCount, метод проверяет наличие активных броней:

```
SELECT COUNT(*) AS Cnt FROM BookReservations
WHERE BookID = @BookID AND Status = 'Active'
```

При наличии броней возвращается WriteOffResult.HasActiveBookings - отображается панель подтверждения. При согласии администратора DataStore.CancelBookingsForBook(bookId) выполняет:

```
UPDATE Books
SET AvailableCount = AvailableCount +
```

```
(SELECT COUNT(*) FROM BookReservations
 WHERE BookID = @BookID AND Status = 'Active')
WHERE BookID = @BookID;
```

```
UPDATE BookReservations SET Status = 'Cancelled'
WHERE BookID = @BookID AND Status = 'Active';
```

6. Успешное списание выполняется двумя SQL-командами: сначала в WriteOffRecords сохраняется JSON-снимок данных книги на момент списания - для возможного восстановления:

```
INSERT INTO WriteOffRecords
 (BookID, AdminID, Reason, SnapshotData, CanBeRestoredUntil)
VALUES
 (@BookID, @AdminID, @Reason, @Snapshot,
  DATEADD(HOUR, 24, GETDATE()));
```

Где @Snapshot - JSON вида:

```
{"BookID":5,"Title":"Sapiens","TotalCount":2,"AvailableCount":2}
```

Затем обновляется таблица Books:

Полное списание (все экземпляры):

```
UPDATE Books
 SET IsArchived = 1, AvailableCount = 0, TotalCount = 0
 WHERE BookID = @BookID;
```

Частичное списание (часть экземпляров):

```
UPDATE Books
 SET TotalCount = TotalCount - @ActualCount,
    AvailableCount = AvailableCount - @ReduceAvailable
 WHERE BookID = @BookID;
```

7. Восстановление из архива: в разделе архива администратор нажимает “Восстановить”. DataStore.RestoreFromRecord(recordId) проверяет CanBeRestoredUntil > GETDATE() и восстанавливает книгу из JSON-снимка.

3. Тестирование работы приложения

Тесты интеграции с MS SQL Server

Тест №1: Сохранение данных после перезапуска приложения

Цель: проверить что данные хранятся в SQL Server, а не в памяти приложения.

Входные данные: зарегистрированный пользователь и добавленная книга.

Шаги:

- Зарегистрировать нового пользователя через форму регистрации.
- Добавить книгу через Admin → AddBook.
- Остановить IIS Express.
- Снова запустить проект.

Ожидаемый результат: книга отображается в каталоге, пользователь может войти под своими данными - данные сохранились в БД и не сбросились.

Результат выполнения программы:

Список книг до добавления:

[OK] найдено: 6 записей					
название	автор	год	жанр	доступно	действия
asd	Автор1	2022	Художественная	0 шт.	[подробнее]
Война и мир	Лев Толстой	1869	Художественная	1 шт.	[подробнее] [забронировать]
Гарри Поттер и философский камень	Джон Роулинг	1997	Детская	2 шт.	[подробнее] [забронировать]
Краткая история времени	Стивен Хокинг	1988	Научная	2 шт.	[подробнее] [забронировать]
Маленький принц	Антуан де Сент-Экзюпери	1943	Детская	1 шт.	[подробнее] [забронировать]
Мастер и Маргарита	Михаил Булгаков	1967	Художественная	3 шт.	[подробнее] [забронировать]

Добавление книги

ДОБАВЛЕНИЕ НОВОГО ЭКЗЕМПЛЯРА

[← НАЗАД В КАТАЛОГ](#)

НАЗВАНИЕ:

Книга1

АВТОР:

Автор1

ISBN:

978-3-16-148510-0

ИЗДАТЕЛЬСТВО:

Издательство1

ГОД:

2026

ЖАНР:

Учебная

СТРАНИЦ:

456

ЭКЗЕМПЛЯРОВ:

1

[>> ДОБАВИТЬ]

Новая книга появилась.

Книга1	Автор1	2026	Учебная	1 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Краткая история времени	Стивен Хокинг	1988	Научная	2 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]

Список книг после обновлений:

[OK] Найдено: 7 записей						
НАЗВАНИЕ	АВТОР	ГОД	ЖАНР	ДОСТУПНО	ДЕЙСТВИЯ	
asd	Автор1	2022	Художественная	0 шт.	[ПОДРОБНЕЕ]	
Война и мир	Лев Толстой	1869	Художественная	1 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Гарри Поттер и философский камень	Джоан Роулинг	1997	Детская	2 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Книга1	Автор1	2026	Учебная	1 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Краткая история времени	Стивен Хокинг	1988	Научная	2 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Маленький принц	Антуан де Сент-Экзюпери	1943	Детская	1 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]
Мастер и Маргарита	Михаил Булгаков	1967	Художественная	3 шт.	[ПОДРОБНЕЕ]	[ЗАБРОНИРОВАТЬ]

Добавленная книга сохранилась.

Тест №2: Проверка уникальности Email на уровне базы данных.

Цель: убедиться что ограничение UNIQUE на поле Email срабатывает на уровне SQL Server, а не только на уровне C#.

Входные данные: Зарегистрированный Email на duplicate@test.com.

Шаги:

1. Зарегистрировать пользователя с Email duplicate@test.com через форму.
2. Попытаться выполнить в SSMS напрямую:

USE BookCatalogDB;

INSERT INTO Users (FullName, Email, PasswordHash)

VALUES (N'Тест', N'duplicate@test.com', N'123');

Ожидаемый результат: SQL Server возвращает ошибку Violation of UNIQUE KEY constraint - дублирование невозможно даже в обход приложения.

Фактический результат:

```
Msg 2627, Level 14, State 1, Line 2
Violation of UNIQUE KEY constraint 'UQ_Users_A9D10534B8F7D596'. Cannot insert duplicate key in object 'dbo.Users'. The duplicate key value is (duplicate@test.com).
The statement has been terminated.
```

Создать аккаунт на ту же почту не получилось.

Тест №3: Транзакционность бронирования

Цель: проверить что sp_CreateBooking работает целостно, либо создаётся бронь и уменьшается AvailableCount, либо ничего не меняется.

Входные данные: книга с AvailableCount = 1, два одновременных запроса

на бронирование.

Шаги:

1. Открыть книгу с 1 доступным экземпляром в двух разных браузерах под разными аккаунтами.
2. Оба пользователя нажимают “Подтвердить бронь” практически одновременно.

После этого выполнить в SSMS:

```
USE BookCatalogDB;  
SELECT AvailableCount FROM Books WHERE BookID = <id книги>;  
SELECT COUNT(*) AS ActiveBookings  
FROM BookReservations  
WHERE BookID = <id книги> AND Status = 'Active';
```

Ожидаемый результат: AvailableCount = 0, ActiveBookings = 1 - только одна бронь создана, вторая получила отказ. Данные целостны.

Результат: у второго пользователя на странице показывается уведомление, что экземпляры закончились, данные не изменились.

Тест №4: Проверка хранимой процедуры sp_SearchBooks

Цель: убедиться что поиск выполняется через хранимую процедуру с параметрами, а не через прямой SQL-запрос.

Входные данные: поиск по автору "Толстой", жанр "fiction", год от 1800 до 1900.

Шаги:

1. Выполнить поиск в интерфейсе с указанными параметрами.
2. Параллельно в SSMS выполнить процедуру напрямую:

```
USE BookCatalogDB;  
EXEC sp_SearchBooks  
    @Author = N'Толстой',  
    @Genre = N'fiction'
```

Ожидаемый результат: результаты в интерфейсе и в SSMS совпадают - отображается "Война и мир".

Результат:

SQL запрос:

	BookID	Title	Author	ISBN	Publisher	Year	Genre	AvailableCount	TotalCount	CoverImageURL
1	15	Война и мир	Лев Толстой	978-5-17-090000-3	Эксмо	1869	fiction	1	1	NULL

Поиск на сайте:

ПОИСК ДАННЫХ

НАЗВАНИЕ:

АВТОР:

ЖАНР:

ГОД:

>> ИНИЦИИРОВАТЬ ПОИСК [X] СБРОС

[ОК] НАЙДЕНО: 1 ЗАПИСЕЙ

НАЗВАНИЕ	АВТОР	ГОД	ЖАНР	ДОСТУПНО	ДЕЙСТВИЯ
Война и мир	Лев Толстой	1869	Художественная	1 шт.	[ПОДРОБНЕЕ] [ЗАБРОНИРОВАТЬ]

Данные совпадают.

Тест №5: Автоматическое истечение брони через SQL

Цель: проверить что `ExpireOldBookings()` корректно обновляет записи в базе данных при загрузке страницы.

Входные данные: создать бронь с истёкшим сроком напрямую через SSMS.

Шаги:

1. В SSMS создать бронь с уже прошедшей датой истечения:

```
USE BookCatalogDB;
```

```
GO
```

```
-- 1. Создаем бронь, которая устарела 2 дня назад
```

```
INSERT INTO BookReservations (UserID, BookID, ReservedAt,  
ExpiresAt, Status)
```

```
VALUES (
```

```
    1, -- UserID (Администратор)
```

```
    13, -- BookID ("Мастер и Маргарита")
```

```
    DATEADD(DAY, -5, GETDATE()), -- Забронировано 5 дней назад
```

```
    DATEADD(DAY, -2, GETDATE()), -- Срок истек 2 дня назад
```

```
    'Active'
```

```
);
```

```
-- 2. Имитируем блокировку книги (уменьшаем доступное кол-во  
вручную)
```

```
-- Если AvailableCount станет 0, кнопка бронирования пропадет в  
интерфейсе
```

```
UPDATE Books
```

```
SET AvailableCount = AvailableCount - 1
```

```
WHERE BookID = 13;
```


2. Открыть главную страницу приложения (Default.aspx).

Проверить в SSMS:

-- Проверяем статус брони

```
SELECT ReservationID, UserID, BookID, Status, ExpiresAt
FROM BookReservations
WHERE UserID = 1 AND BookID = 13;
```

-- Проверяем доступность книги

```
SELECT BookID, Title, AvailableCount
FROM Books
WHERE BookID = 13;
```

Ожидаемый результат: статус брони изменился на Expired, AvailableCount книги восстановлен - ExpireOldBookings() вызывается при каждом Page_Load на Default.aspx и обновляет данные в БД.

Вывод:

Статус книги изменился на Expired

	ReservationID	UserID	BookID	Status	ExpiresAt
1	12	1	13	Expired	2026-04-10 15:25:37.497

Количество экземпляров вернулось обратно:

	BookID	Title	AvailableCount
1	13	Мастер и Маргарита	3

Выводы:

В ходе выполнения лабораторной работы веб-приложение "Книжный каталог" было успешно доработано.

- Реализован переход от in-memory хранилища к полноценной СУБД MS SQL Server: создана схема базы данных с пятью таблицами (Users, Books, BookReservations, WriteOffRecords, индексы), разработаны шесть хранимых процедур (sp_RegisterUser, sp_AuthenticateUser, sp_SearchBooks, sp_CreateBooking, sp_AddBook, sp_WriteOffBook) с поддержкой транзакций и параметризованных запросов.
- Все пять вариантов использования из лабораторной работы №1 реализованы и протестированы в полном объеме, включая граничные случаи: проверку дубликатов ISBN, лимит бронирований на пользователя, отмену брони при списании книги, восстановление книг из архива в течение 24 часов, автоматическое

истечение срока действия броней.

- Продemonстрировано применение всех шести принципов проектирования пользовательского интерфейса: единообразие, обратная связь, видимость, предотвращение ошибок, восстановление после ошибок, контроль пользователя.